Zero Trust Network Access (ZTNA) with Zscaler: Securing User-to Application Connectivity.

PROJECT DETAILS

Title: Zero Trust Network Access (ZTNA) with Zscaler: Securing User-to-Application Connectivity

AI-Enhanced User Protection (AIUP)

AI-Enhanced User Protection (AIUP) is an advanced security framework that utilizes artificial intelligence and machine learning techniques to authenticate users and safeguard online platforms from cyber threats. Unlike conventional security systems that depend on static authentication methods such as passwords or two-factor authentication (2FA), AIUP continuously monitors and analyses user behaviour to generate a dynamic and evolving trust score for each user.

AIUP plays a vital role in modern cybersecurity as it overcomes the limitations of traditional perimeter-based security models. With cyber threats becoming increasingly sophisticated—such as attacks involving stolen credentials, phishing, and social engineering—relying only on “what a user knows” (passwords) or “what a user possesses” (authentication tokens) is no longer adequate. AIUP introduces an additional layer of protection based on “who the user is”, determined through behavioral patterns and interactions with the system. By continuously verifying user behaviour during a session, AIUP helps organizations prevent account hijacking, minimize fraudulent activities, and ensure that the user accessing the platform is genuinely the legitimate account owner.

Abstract

The limitations of traditional VPN architectures in handling modern cybersecurity threats, particularly issues such as lateral movement and excessive network privileges, highlight the need for a transition toward Zero Trust security models. This project presents the design, implementation, and management of a Zero Trust Network Access (ZTNA) solution using Zscaler Private Access (ZPA) within a large-scale institutional network environment. The primary objective of the project is to replace broad network-level access with application-specific and identity-based connectivity.

The proposed architecture involves deploying Zscaler App Connectors within the organization’s data center and integrating the system with existing Identity Providers such as Azure Active Directory (Azure AD) along with endpoint security tools. Through this configuration, all connections between users and internal applications are established through a cloud-native security broker, ensuring secure and controlled access. This model effectively makes internal applications invisible to the public internet, thereby significantly reducing the attack surface.

Use Case Scenarios

Scenario 1: Protected Remote Access to the ERP System for Faculty: A faculty member working remotely needs access to the university's ERP system to upload student grades. Through Zscaler Private Access (ZPA), the connection is established securely without granting access to the internal network. The system authenticates the user using Azure AD multi-factor authentication (MFA) and verifies the security status of the device before permitting access.

Output:

- Faculty identity successfully authenticated (Faculty_Group)
- Device security posture validated (antivirus enabled, operating system updated)
- Secure micro-tunnel created specifically for the ERP application
- Internal network remains hidden from the user

Scenario 2:Temporary Database Access for External Research Partners:

A research collaborator from another institution requires temporary access to a particular research database hosted on the university's servers. The administrator configures a time-limited and application-specific access policy in Zscaler. The researcher is able to access only the authorized database, with no visibility into other parts of the network.

Output:

- External researcher authenticated using temporary credentials
- Access restricted by policy (7-day duration, Research-DB only)
- Database server protected from internet scanning
- Session automatically terminated once the access period expires

---

Scenario 3: Student Authentication for the Online Examination System: During final examinations, a large number of students simultaneously access the university's online examination portal from different locations. Zscaler ZPA ensures that only authenticated students with compliant devices can reach the exam server. The system also prevents potential DDoS attacks and blocks attempts to access other internal systems.

Output:

- High-volume authentication supported through cloud scalability
- Students granted access solely to the examination portal
- Unauthorized access to other internal resources prevented
- Exam server IP address remains concealed from the internet

Scenario 4:Controlled Administrative Access for Server Maintenance:

An IT administrator needs to carry out maintenance on an internal application server. Instead of using a VPN that provides full network access, the administrator connects directly to the required server interface using ZPA. The session is monitored continuously, and any suspicious activity—such as attempts to access unauthorized systems—is immediately blocked.

Output:

- Administrator identity verified through MFA
- Direct micro-tunnel established only to the server management interface
- All administrative activities recorded for audit purposes
- Unauthorized server access attempts automatically denied

Scenario 5:Temporary System Access for Contractors During Upgrade:

A contractor hired for a university system upgrade requires short-term access to a specific file server for data migration. Using Zscaler ZPA, the contractor receives restricted access to the required resource without being added to the university's Active Directory or receiving VPN credentials. The access automatically expires once the project is completed.

Output:

- Contractor identity verified through a separate identity provider
- Access limited to the designated file server migration share
- All file transfer activities logged in Zscaler and the SIEM system
- Access revoked automatically after the 30-day project period

Stage-1

Title Of the Project: -

Zero Trust Network Access (ZTNA) with Zscaler: Securing User-to-Application Connectivity.

Overview

The objective of this project is to develop an AI-based identity verification system that strengthens online security by analyzing and validating user identities through their behavioral patterns on digital platforms. The system applies advanced machine learning techniques to build detailed user profiles, enabling it to differentiate between legitimate users and suspicious activities. By doing so, it helps address critical issues such as unauthorized access, fraudulent transactions, and various cyber threats affecting online services.

The main goal of the system is to improve security by continuously monitoring and verifying user identities based on their behavior. Advanced technologies such as deep learning, neural networks, and natural language processing (NLP) are utilized to generate dynamic behavioral profiles for each user. These profiles adapt and evolve over time as user behavior changes. A key component of the system is real-time anomaly detection, which identifies unusual activities that deviate from normal behavioral patterns and immediately responds to reduce potential security risks.

To further strengthen protection, the system incorporates multi-factor authentication, which triggers additional verification steps whenever suspicious behavior is detected. Both users and administrators receive alerts to maintain transparency and assist in resolving potential security concerns. The system also includes secure mechanisms for storing and managing sensitive user data and anomaly records.

An integration layer allows the system to interact seamlessly with different online platforms and services. Monitoring and reporting features provide insights into system performance and security status, ensuring transparency and accountability. The architecture is designed to be scalable, with load balancing mechanisms that support increasing numbers of users. Security and regulatory compliance are ensured through encryption of data both in transit and at rest, along with the use of firewalls and intrusion detection systems.

Additionally, the system can be deployed using cloud infrastructure to enhance flexibility and scalability. Continuous training and optimization of machine learning models help maintain effectiveness against evolving cyber threats and changing user behaviors. Deploying system components across multiple geographic regions can further improve redundancy, reliability, and overall system performance.

List Of Vulnerability:

Practice test:

S.NO	Vulnerability Name	CWE-No
1	Improper Access Control	CWE-284
2	Active Debug Code	• CWE-489
3	Use of Obsolete Function	• CWE-447
4	Weak Encoding for Password	• CWE-261
5	Improper Input Validation	• CWE-20
6	External Control of File Name or Path	• CWE-73
7	Use of Hard-coded Password	• CWE-259
8	Untrusted Search Path	• CWE-426
9	Information Loss of Omission	• CWE-221
10	Uncaught Exception	• CWE-248

REPORT: -

Practice Website: https://cwe.mitre.org/data/definitions/276.html

TGTCWE- 284: Improper Access Control

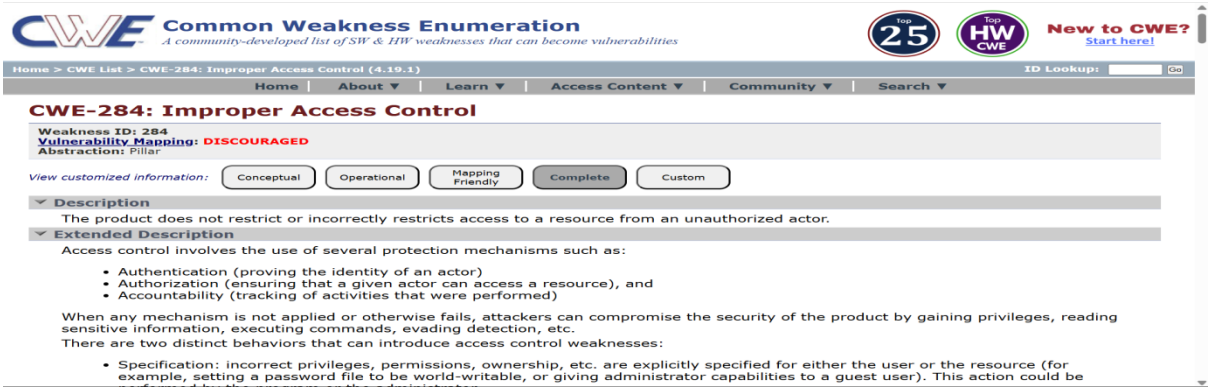
OWASP Category: A01:2021-Broken Access Control

Description: The vulnerability occurs when a system or application fails to properly limit access to its resources, allowing unauthorized users to view, modify, or interact with sensitive information or functions. This improper restriction can lead to security risks such as unauthorized access, data exposure, or misuse of system resources.

BusinessImpact:

This vulnerability is critical because it may allow unauthorized individuals to access sensitive data or files. Such exposure can result in data breaches, privacy violations, and damage to user trust. In addition, affected systems may need configuration updates or changes after implementing the required security patches or fixes.

Steps to Reproduce:https://www.shodan.io/host/8.8.8.8



Vulnerability Name: Active Debug Code

CWE-489: Active Debug Code

OWASP Category:**A05:2021 – Security Misconfiguration**

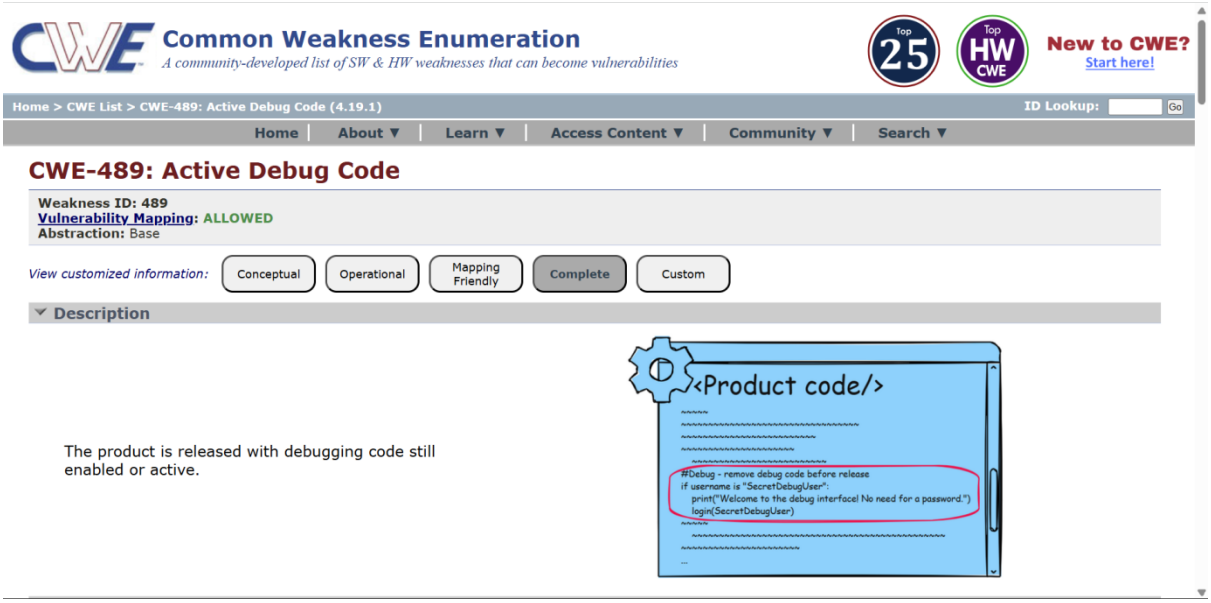
Description:

Active Debug Code refers to debugging functionality that remains enabled in the production environment. Debug code is usually added by developers during the development or testing phase to identify errors and monitor system behavior. If this code is not removed before deployment, it may expose sensitive information such as internal application logic, configuration details, system paths, or administrative access functions. Attackers can exploit

these debugging interfaces to gain unauthorized access or understand the internal workings of the application.

Business Impact:

The presence of active debug code in a live system can lead to serious security risks. Attackers may use debugging interfaces to bypass authentication, retrieve sensitive system information, or execute unauthorized commands. This may result in data breaches, system compromise, financial loss, and reputational damage to the organization. Additionally, exposing debug functions may allow attackers to manipulate application behavior or gain administrative privileges.



Vulnerability Path:

http://testphp.vulnweb.com/debug

Steps to Reproduce:

- Open the target website in a web browser.
- Navigate to the application URL and attempt to locate debugging endpoints or parameters such as /debug**,** /test**, or** /admin/debug.
- Inspect the page source or application responses for debugging messages or developer comments.
- If debug information such as system logs, stack traces, or developer notes are displayed, it indicates that debug code is active.
- Observe whether sensitive system information is revealed through the debug interface.

Vulnerability Name: Use of Obsolete Function

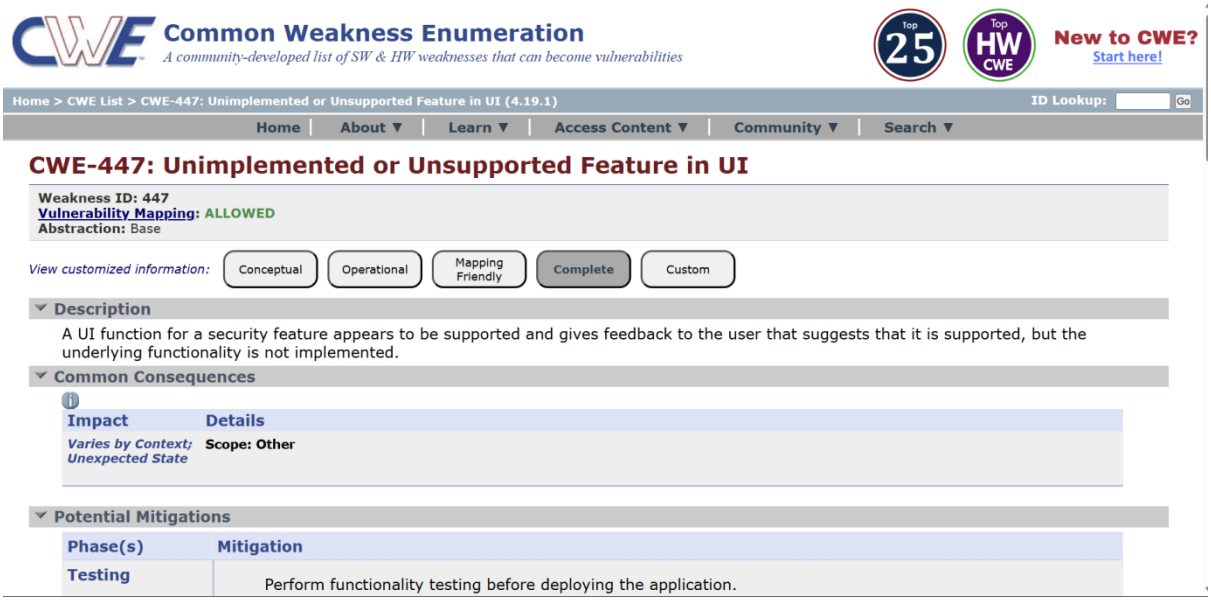
CWE ID: CWE-447 – Use of obsolete Function
OWASP Category: A03:2021 – Software Supply Function

Description:

It occurs when a security-related feature is advertised or expected to be present in a system but is not actually implemented or supported. This creates a false sense of security because administrators or users may assume that a specific security control is active when it is not. Attackers can exploit this weakness by bypassing the missing protection mechanism, allowing them to perform actions that should have been blocked by the security feature.

Business Impact:

If a security feature is missing or not implemented properly, the system may remain exposed to attacks. Organizations may rely on the feature for protection against threats such as unauthorized access, malware, or data leakage. When the feature does not function as expected, attackers may exploit the gap to compromise systems, steal sensitive data, or disrupt services. This can lead to operational disruption, financial loss, and reputational damage.



Vulnerability Path:

`http://testphp.vulnweb.com/security-settings`

Steps to Reproduce:

- Open the target web application in a browser.

- Identify security features advertised by the application (for example, authentication protection, logging, or input validation).

- Attempt to perform actions that should normally be restricted by the security feature.

• Monitor the system response using browser developer tools or security testing tools.

• If the system allows the action without enforcing the expected security control, the feature may not be properly implemented or supported.

Vulnerability Name: Weak Encoding for Password

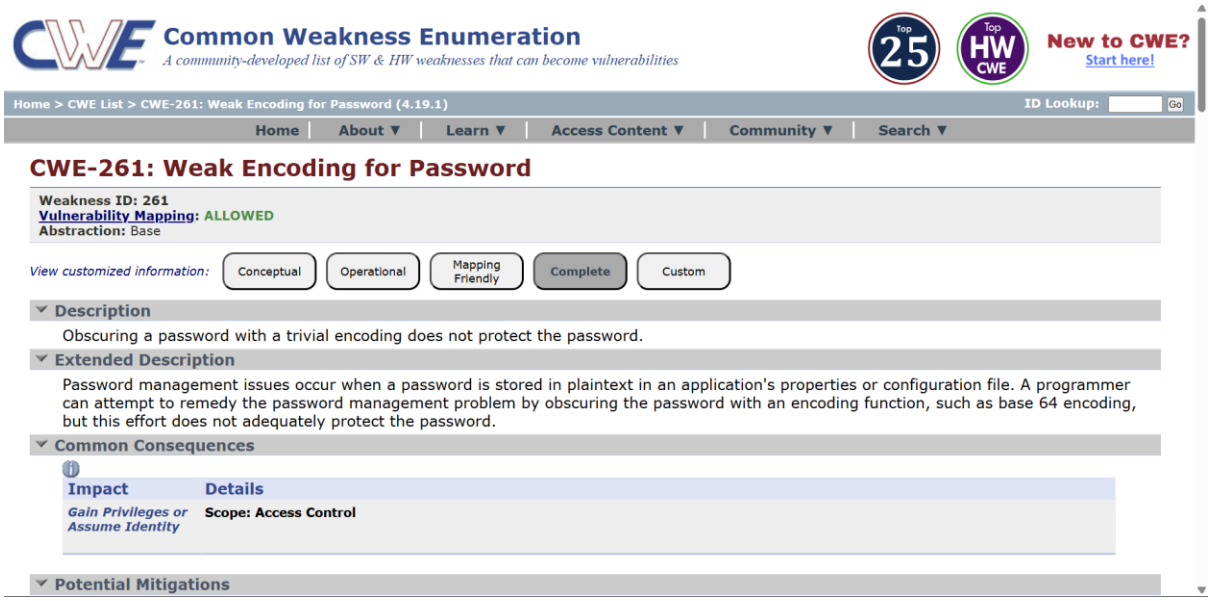
CWE ID: CWE-261-Weak Encoding for Password
OWASP Category: A04:2025–Cryptographic Failures

Description:

Weak Encoding for Password occurs when a password is protected using simple encoding techniques such as Base64 instead of secure encryption or hashing. Encoding methods are reversible and do not provide real security for sensitive information. If attackers gain access to encoded passwords, they can easily decode them to retrieve the original password. This vulnerability usually occurs when developers attempt to hide passwords in configuration files or databases using weak encoding mechanisms rather than strong cryptographic protection.

Business Impact:

If passwords are weakly encoded instead of securely hashed or encrypted, attackers who access the system or database can easily decode them. This can lead to unauthorized access to user accounts, privilege escalation, and compromise of sensitive systems. Stolen credentials may also allow attackers to access other systems if users reuse the same passwords. This can result in data breaches, financial loss, and damage to organizational reputation.



Vulnerability Path:

`http://testphp.vulnweb.com/login.php`

Steps to Reproduce:

1. Open the target web application in a browser.
2. Attempt to create or log in to an account.
3. Capture the request using tools such as **Burp Suite** or **browser developer tools**.
4. Inspect the stored password in the application database or configuration files.
5. If the password is encoded using simple encoding (such as **Base64**) instead of hashing or encryption, decode it using an online decoder.
6. If the original password can be retrieved easily, the vulnerability exists.

Solution

- Store passwords using strong hashing algorithms such as bcrypt, Argon2, or PBKDF2**.**
- Avoid using simple encoding techniques such as Base64 for password protection.
- Implement salting to strengthen password hashing.
- Restrict access to configuration files and sensitive data.
- Perform regular security audits and code reviews to ensure secure password storage practices.

Vulnerability Name: Improper Input Validation

CWE ID: CWE-285 – Improper Authorization
OWASP Category: A05:2025 – Injection

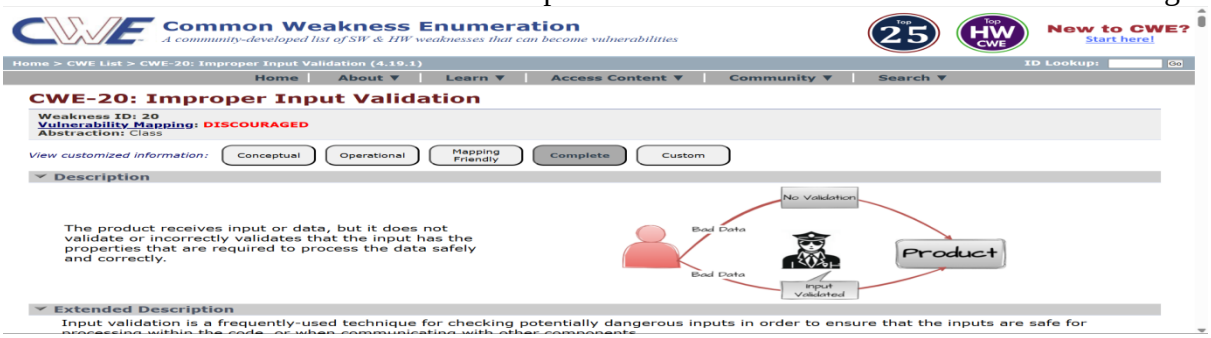
Description:

It occurs when an application receives data from an external source but fails to verify that it has the properties required to process it safely. Instead of checking for correct type, length, or syntax, the system trusts the "Bad Data" (as seen in the diagram), allowing it to reach sensitive components. This lack of a "security filter" is a primary root cause for most injection-based cyberattacks.

Business Impact:

Improper validation leads to **data breaches** and **unauthorized system access**, which can result in heavy regulatory fines and legal liabilities. Operationally, it allows attackers to trigger system crashes or "Denial of Service" (DoS) attacks, causing expensive downtime. Ultimately, a single unvalidated input can lead to a total loss of customer trust and severe

brand reputation damage.



Vulnerability Path: http://www.itsecgames.com/

Steps to Reproduce:

- **Identify:** Locate an input field, such as a "Quantity" box or a URL parameter (e.g., `?id=10`).
- **Intercept:** Use a browser inspector or proxy tool (like Burp Suite) to capture the request.
- **Modify:** Change the legitimate value to an "illegal" one, such as a negative number (`-500`) or a script tag (`<script>`).
- **Submit:** Send the modified request to the server.
- **Verify:** Observe if the system processes the invalid data (e.g., calculates a negative price) instead of returning an error.

Solution

- **Strict Whitelisting:** Only allow data that matches a known "good" pattern (e.g., regex for alphanumeric only).
- **Server-Side Validation:** Never rely on client-side (browser) checks, as they are easily bypassed.
- **Type Checking:** Ensure input matches the expected data type (e.g., an integer cannot be a string).
- **Range & Length Limits:** Enforce minimum/maximum constraints on all numerical and text inputs.
- **Parameterized Queries:** Use secure APIs to ensure input is treated as data, not executable code.

Vulnerability Name: External Control Of File Name or Path

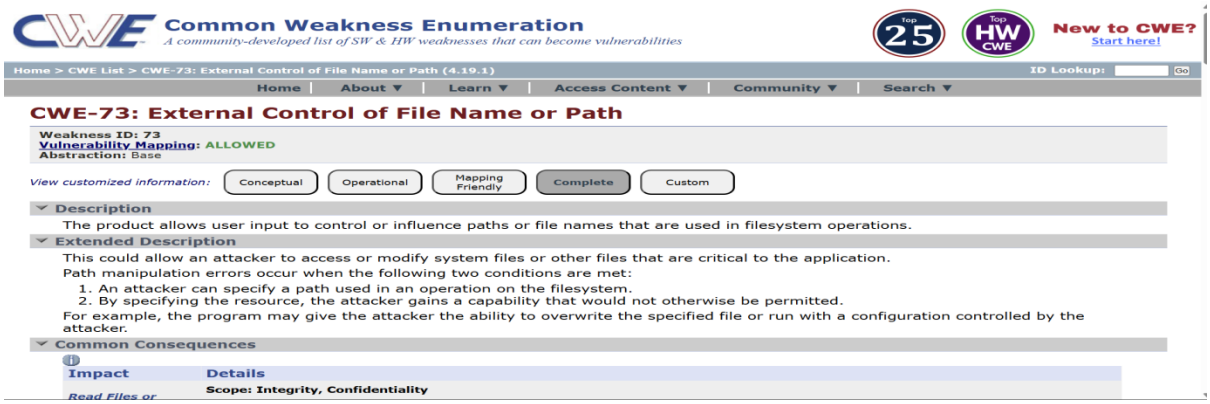
- **CWE ID:** CWE-73 –External Control Of File Name or Path **OWASP Category:** A06:2025 – Insecure Design

Description:

CWE-73 occurs when an application allows user-supplied input to directly influence or control the file paths and names used in filesystem operations. By failing to restrict where a file can be read from or written to, the software creates a "Path Manipulation" vulnerability. This allows an attacker to bypass intended directory restrictions and interact with sensitive system files that should be inaccessible to them.

Business Impact:

The primary impact involves the total compromise of **confidentiality and integrity**. Attackers can read sensitive configuration files (like `/etc/passwd` or `.env` files) containing credentials, or overwrite critical system binaries to gain persistent access. For a business, this leads to catastrophic data leaks, potential service outages due to deleted files, and severe compliance violations regarding the protection of sensitive data.



Vulnerability Path:

http://www.itsecgames.com/

Steps to Reproduce:

- **Identify:** Locate a feature that loads or saves files based on a user parameter (e.g., `download.php?file=report.pdf`).
- **Intercept:** Use a proxy tool to capture the request before it reaches the server.
- **Modify:** Replace the filename with a traversal sequence, such as `../../../../etc/passwd` (on Linux) or `..\..\..\windows\win.ini` (on Windows).
- **Submit:** Send the modified request to the server.
- **Verify:** Check if the application returns the contents of the sensitive system file instead of the intended report.

Solution

- **Use an Indirect Map:** Instead of letting users provide names, use an ID or index (e.g., `file=123`) that maps to a hardcoded path in a secure database.
- **Path Validation:** Ensure the final resolved path starts with the expected "base" directory and does not contain `../` or `..\\` sequences.
- **Hardcoded Paths:** Whenever possible, keep the file directory hardcoded and only allow the user to select from a strict whitelist of filenames.
- **Least Privilege:** Run the application with a low-privilege service account that has no access to sensitive system directories.
- **Chroot Jail:** Use a "jail" or container environment to logically isolate the application's filesystem from the rest of the OS.

Vulnerability Name: Use Of Hard-coded Password

CWE ID: CWE-259 –Use of Hard Coded Password

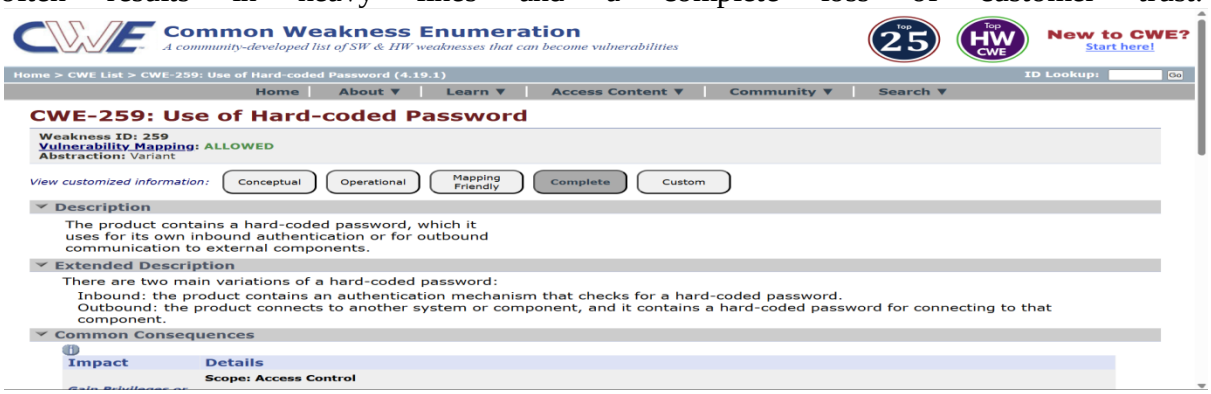
OWASP Category: A07:2025 – Authetication Failures

Description:

CWE-259 occurs when an application contains a password embedded directly into its source code or configuration files. This password is used for either **inbound** authentication (checking a user's password against a static string) or **outbound** communication (automatically logging into a database or external service). Because the password is hard-coded, it cannot be easily changed without modifying the software, and it is visible to anyone with access to the compiled binary or the source repository.

Business Impact:

The business impact is severe as it creates a permanent "backdoor" into the system. If the hard-coded credential is discovered, an attacker can gain **unauthorized administrative access** to databases, servers, or third-party APIs. This leads to massive data breaches, loss of intellectual property, and non-compliance with security standards (like PCI-DSS), which often results in heavy fines and a complete loss of customer trust.



Vulnerability Path:

http://www.itsecgames.com/

Steps to Reproduce:

- **Identify:** Look for authentication prompts or connections to external services (like a database or an API).
- **Analyze:** Perform a "grep" search in the source code for keywords like `password`, `pwd`, or `connectionString`.
- **Reverse Engineer:** If source code isn't available, use a decompiler or strings utility on the application binary to look for plain-text strings that appear to be passwords.
- **Attempt Access:** Use the discovered password to log in through the application's administrative interface or via a direct connection to the backend service.
- **Verify:** Confirm that the static password works across different installations or environments of the same software.

Solution

- **External Secret Management:** Store passwords in secure, encrypted vaults like AWS Secrets Manager, HashiCorp Vault, or Azure Key Vault.
- **Environment Variables:** Use system-level environment variables to inject credentials at runtime rather than storing them in the code.
- **Password Hashing:** For inbound authentication, never store the actual password; only store and compare a high-entropy, salted hash (e.g., Argon2 or bcrypt).
- **Dynamic Credentials:** Use temporary, short-lived tokens or Managed Identities instead of static, long-term passwords.
- **Code Scanning:** Integrate Static Application Security Testing (SAST) tools into the CI/CD pipeline to automatically detect hard-coded strings before they are deployed.

Vulnerability Name: Untrusted Search Path

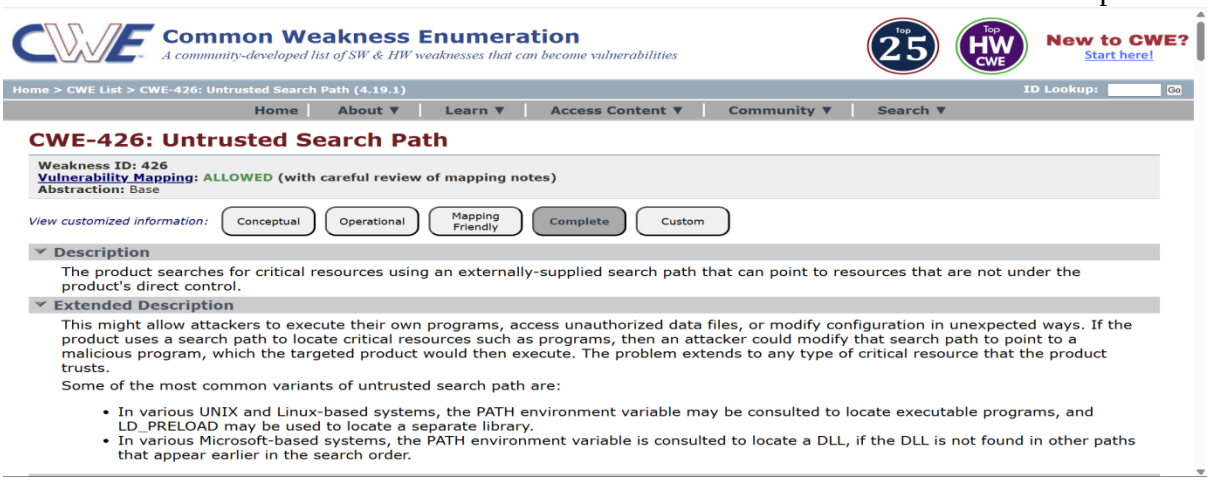
CWE ID: CWE-426–Untrusted Search Path
OWASP Category: A08:2025 – Broken Access Control

Description:

CWE-426 occurs when an application searches for a critical resource—such as a dynamic link library (DLL), an executable program, or a configuration file—using an externally-supplied search path that is not under the application's direct control. If the application relies on an environment variable (like `PATH` or `LD_LIBRARY_PATH`) or a relative directory to find these resources, it may inadvertently load a malicious version of that resource placed there by an attacker. This is a common root cause for "Binary Planting" or "DLL Preloading" attacks.

Business Impact:

The business impact is severe, as this vulnerability often leads to **Local Privilege Escalation (LPE)**. If an application with administrative or "root" privileges loads a malicious library, the attacker gains full control over the underlying operating system. This can result in the installation of persistent malware, the theft of sensitive corporate data, or the complete disruption of business services, often going undetected as the malicious code runs under a "trusted" process.



Vulnerability Path:
http://www.itsecgames.com/

Steps to Reproduce:

- **Identify:** Locate an application that calls external commands (e.g., `system("grep ...")`) or loads libraries without using absolute paths.
- **Analyze Search Order:** Determine the search order of the OS (e.g., does it check the current working directory before system folders?).
- **Craft Payload:** Create a malicious executable or DLL and name it exactly like the resource the application is looking for (e.g., `grep.exe` or `library.so`).
- **Place Payload:** Drop this malicious file into a directory that appears earlier in the search path than the legitimate file.
- **Verify:** Run the target application; if the application executes your malicious file instead of the system original, the vulnerability is confirmed.

Solution

- **Use Absolute Paths:** Always specify the full, hardcoded path to an executable or library (e.g., use `/usr/bin/grep` instead of just `grep`).
- **Sanitize Environment Variables:** Before loading resources, clear or reset sensitive environment variables like `PATH`, `LD_PRELOAD`, or `PYTHONPATH` to a known safe state.
- **Strict Search Order:** Programmatically set the search order to ignore the current working directory (e.g., use `SetDefaultDllDirectories` on Windows).
- **Verify Signatures:** Implement cryptographic signature checking to ensure that any library or executable being loaded is signed by a trusted vendor.
- **Least Privilege:** Ensure the application runs with the minimum permissions necessary, limiting the damage an attacker can do if they successfully plant a file.

Vulnerability Name: Information Loss Of Omission

CWE ID: CWE-221 – Information Loss Of Omission
OWASP Category: A09:2025 – Security Logging And Alerting failures

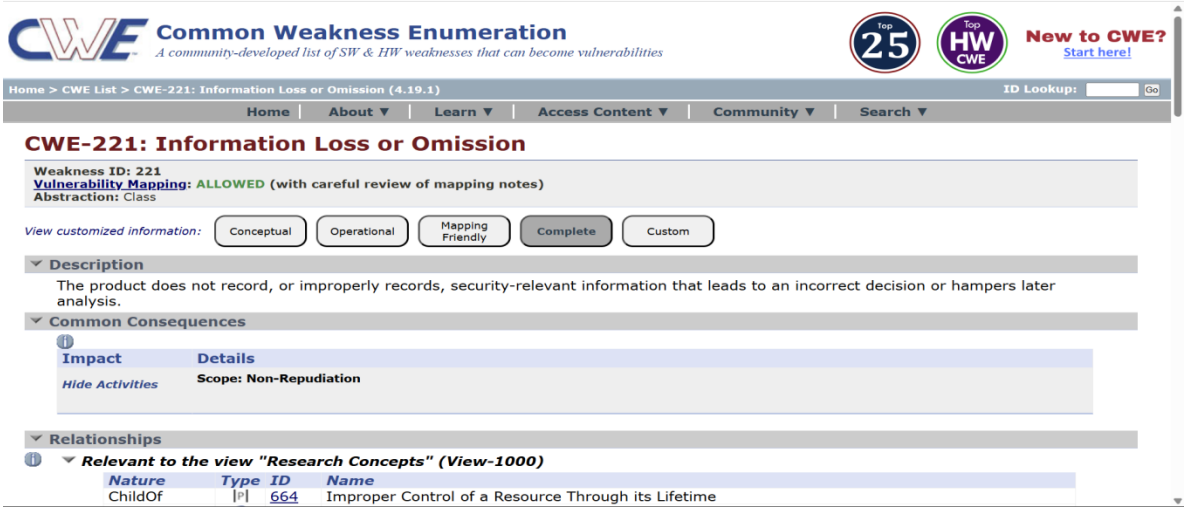
Description:

CWE-221 occurs when a product fails to record, or improperly records, security-relevant information. This omission can lead to incorrect automated decisions by the system or significantly hamper manual analysis by security administrators. When critical details—such as user identity, origin of a request, or the outcome of a sensitive operation—are lost or omitted during processing, the system loses the context necessary to maintain a secure state or provide an accurate audit trail.

Business Impact:

The primary business impact is a failure of **Non-Repudiation**. If security-relevant information is missing, an organization cannot prove that a specific user performed a specific action, making it nearly impossible to hold malicious actors accountable. Furthermore, during an incident response, the lack of detailed logs ("Information Omission") prevents forensic investigators from understanding the scope of a breach, leading to longer recovery times,

increased legal liability, and failure to meet regulatory compliance requirements like SOC2 or HIPAA.



Vulnerability Path: http://testphp.vulnweb.com/

Steps to Reproduce:

- **Identify:** Locate a security-critical function, such as a login attempt, password change, or sensitive data export.

- **Trigger:** Execute the function while intentionally providing edge-case data (e.g., an extremely long username or a failed login from a new IP).

- **Inspect Logs:** Access the system's audit logs or administrative console to see what was recorded.

- **Compare:** Check if critical metadata is missing, such as the timestamp, the source IP address, or the specific reason for a failure.

- **Verify:** Confirm that a security analyst would be unable to reconstruct the timeline of events based solely on the recorded information.

Solution

- **Comprehensive Logging Policy:** Implement a standard logging framework that automatically captures Who, What, Where, When, and Why for every security event.

- **Fail-Secure Mechanisms:** Ensure that if a logging or recording operation fails, the primary sensitive action is also blocked to prevent "silent" failures.

- **Standardized Formats:** Use structured logging (e.g., JSON) to ensure that security-relevant fields are always present and easily searchable.

- **Audit Trail Protection:** Store logs in a centralized, write-once-read-many (WORM) environment to prevent attackers from deleting evidence of their activities.

- **Context Retention:** Ensure that when data is passed between different components of a system, all security-relevant metadata is preserved and not stripped away.

Vulnerability Name: Uncaught Exception

CWE ID: CWE-248 – Uncaught Exception

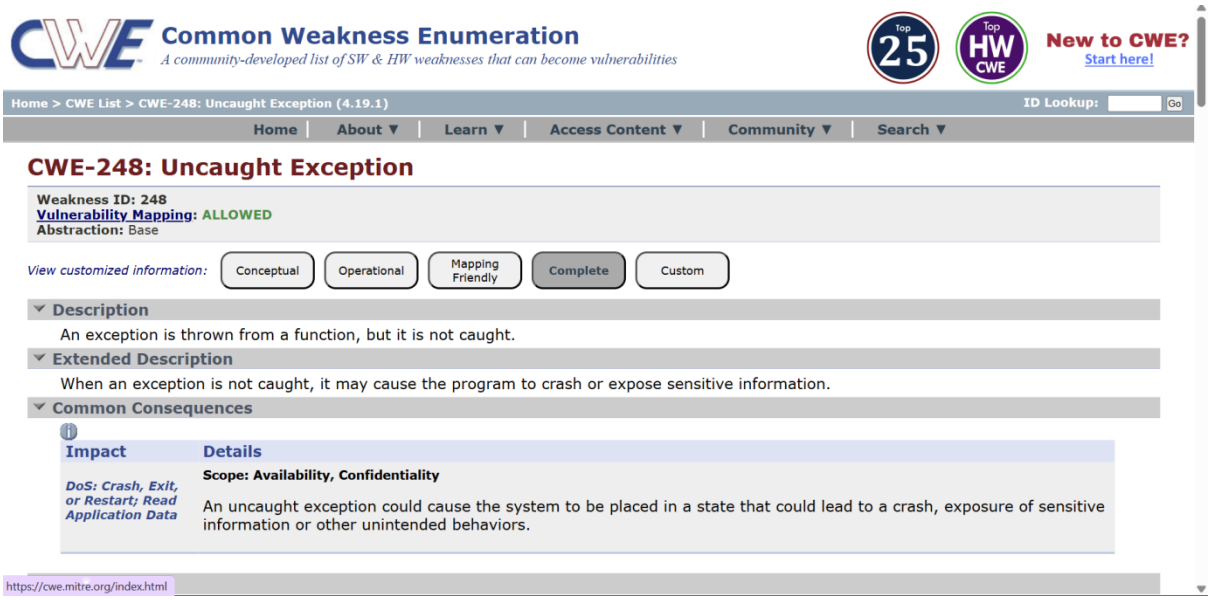
OWASP Category: A10:2025 –Mishandling Of Exceptional Conditions

Description:

loses the context necessary to maintain a secure state or provide an accurate audit trail.

Business Impact:

The primary business impact is a failure of **Non-Repudiation**. If security-relevant information is missing, an organization cannot prove that a specific user performed a specific action, making it nearly impossible to hold malicious actors accountable. Furthermore, during an incident response, the lack of detailed logs ("Information Omission") prevents forensic investigators from understanding the scope of a breach, leading to longer recovery times, increased legal liability, and failure to meet regulatory compliance requirements like SOC2 or HIPAA.



VulnerabilityPath:

http://testphp.vulnweb.com/

Steps to Reproduce:

- **Identify:** Locate a security-critical function, such as a login attempt, password change, or sensitive data export.

- **Trigger:** Execute the function while intentionally providing edge-case data (e.g., an extremely long username or a failed login from a new IP).

- **Inspect Logs:** Access the system's audit logs or administrative console to see what was recorded.

- **Compare:** Check if critical metadata is missing, such as the timestamp, the source IP address, or the specific reason for a failure.

- **Verify:** Confirm that a security analyst would be unable to reconstruct the timeline of events based solely on the recorded information.

Solution

- **Comprehensive Logging Policy:** Implement a standard logging framework that automatically captures Who, What, Where, When, and Why for every security event.

- **Fail-Secure Mechanisms:** Ensure that if a logging or recording operation fails, the primary sensitive action is also blocked to prevent "silent" failures.

- **Standardized Formats:** Use structured logging (e.g., JSON) to ensure that security-relevant fields are always present and easily searchable.

- **Audit Trail Protection:** Store logs in a centralized, write-once-read-many (WORM) environment to prevent attackers from deleting evidence of their activities.

- **Context Retention:** Ensure that when data is passed between different components of a system, all security-relevant metadata is preserved and not stripped away.

Stage-2

Overview

Nessus is a vulnerability scanning tool developed by Tenable, Inc. It is a remote scanner, which means it does not need to be installed on the target system in order to perform scans. Instead, Nessus sends different types of probes to the target host to detect possible security vulnerabilities. These probes may include network traffic checks, operating system commands, and application-level requests.

Nessus operates using a plugin-based architecture. Plugins are small programs created to detect specific types of vulnerabilities. There are more than 1200 plugins available, covering vulnerabilities found in operating systems, applications, and network devices.

During a scan, Nessus first uses a discovery plugin to identify the operating system and the services running on the target host. After identifying the system, Nessus executes multiple plugins to test for potential vulnerabilities. When a vulnerability is detected, Nessus generates a detailed report that includes the severity level, steps to reproduce the issue, and recommended remediation steps.

Nessus can scan various types of targets such as individual hosts, entire networks, and web applications. It also allows scanning of specific ports and services. Scans can be scheduled to run automatically or executed manually based on the user's requirements.

Step-by-Step Working of Nessus

1. Nessus connects to the target host using protocols such as TCP, UDP, and ICMP.
2. It sends multiple probes to identify the operating system and running services on the host.
3. Nessus then runs different plugins to check for known vulnerabilities.
4. After the scan, Nessus generates a detailed report containing the severity of each vulnerability, how the vulnerability can be reproduced, and recommended solutions to fix it.

Benefits of Using Nessus

- **Wide vulnerability coverage:** Nessus includes more than 1200 plugins that detect vulnerabilities in operating systems, applications, and network devices.
- **User-friendly interface:** The tool is easy to configure and operate, making it suitable for security professionals.
- **Detailed reporting:** Nessus provides comprehensive reports that include vulnerability severity, reproduction steps, and remediation guidelines.
- **Scalability:** It can scan different types of environments such as single hosts, networks, and web applications.
- **Flexible scanning options:** Users can perform scans either manually or schedule them at specific times.

- **Saves time and effort:** Automated scanning reduces manual security testing and helps security teams quickly identify and prioritize issues.

Nessus is a powerful and reliable tool that helps organizations identify and fix security vulnerabilities in their systems. It plays an important role in improving overall network security and is widely used by cybersecurity professionals.

Target website :- http://testfire.net/

Target Ip address:- 65.61.137.117

List of vulnerability:-

S.no	Vulnerability name	Severity	Plugins
1.	HTTP Server Type and Version	INFO	10107
2.	Hypertext Transfer Protocal (HTTP) Information	INFO	24260
3.	Nessus SYN scanner	INFO	11219
4.	Nessus Scan Information	INFO	19506

5.	OS Fingerprints Detected	INFO	209654
6.	Service Detection	INFO	22964
7.	Traceroute Information	INFO	10287
8.	Web Server No 404 Error Code Check	INFO	10386

REPORT: -

1.Vulnerability Name: - 10107- HTTP Server Type and Version

Severity:- INFO

Plugin:- 10107

Port:- 80 / tcp / www

Description:-

This plugin attempts to determine the type and the version of the remote web server.

Solution:-

Configure the web server to hide or disable the disclosure of server type and version information in HTTP response headers. This can be done by modifying the web server configuration (such as disabling the ServerTokens or ServerSignature settings). Regularly update and patch the web server to the latest secure version to reduce potential security risks.

BusinessImpact:

If server type and version information is exposed, attackers can easily identify the software

being used and search for known vulnerabilities related to that version. This may increase the risk of targeted cyberattacks, unauthorized access, or service disruption, which could affect business operations and compromise system security.

2.Vulnerability Name: - 24260-Hypertext Tranfer Protocal (HTTP) Information

Severity:- INFO

Plugin:- 24260

Port:- 80 / tcp / www

Description

This test gives some information about the remote HTTP protocol - the version used,whether HTTP Keep-Alive is enabled, etc...
This test is informational only and does not denote any security problem.

Solution:-

Since this finding is informational and does not indicate a direct security vulnerability, no immediate fix is required. However, organizations should ensure that the web server is properly configured, unnecessary HTTP methods are disabled, and security best practices such as regular patching, secure headers, and proper access controls are implemented.

BusinessImpact:

This information alone does not pose a direct security risk. However, if combined with other disclosed system details, attackers may use it to gather information about the server environment and plan potential attacks. Proper configuration and monitoring help reduce the possibility of misuse.

3.Vulnerability Name: - 11219-Nessus SYN Scanner

Severity:- INFO

Plugin:- 11219

Port:- 80 / tcp / www

Description:-

This plugin is a SYN 'half-open' port scanner. It shall be reasonably quick even against a firewalled target.
Note that SYN scans are less intrusive than TCP (full connect) scans against broken services, but they might cause problems for less robust firewalls and also leave unclosed connections on the remote target, if the network is loaded.

Solution:-

Protect your target with an IP filter.

BusinessImpact:-

port scanning can allow attackers to discover open ports and services running on the system. This information may help them identify potential entry points for attacks. If not properly monitored or restricted, it could increase the risk of unauthorized access or exploitation of exposed services.

4.Vulnerability Name: - 19506-Nessus Scan Information

Severity:- INFO

Plugin:- 19506

Port:- N/A

Description

This plugin displays, for each tested host, information about the scan itself :

- The version of the plugin set.
- The type of scanner (Nessus or Nessus Home).
- The version of the Nessus Engine.
- The port scanner(s) used.
- The port range scanned.
- The ping round trip time
- Whether credentialed or third-party patch management checks are possible.
- Whether the display of superseded patches is enabled
- The date of the scan.
- The duration of the scan.
- The number of hosts scanned in parallel.
- The number of checks done in parallel.

Solution:-

This finding is informational and does not represent a vulnerability. No specific remediation is required. However, organizations should securely store and manage scan reports, restrict access to authorized personnel, and ensure vulnerability scans are performed regularly to monitor system security.

BusinessImpact:-

There is no direct security impact because this plugin only provides details about the Nessus

scan configuration and process. However, proper management of scan information helps organizations track security assessments, improve vulnerability management, and maintain a strong security posture.

5.Vulnerability Name: - 209654-OS Fingerprints Detected

Severity:- INFO

Plugin:- 209654

Port:- N/A

Description:-

Using a combination of remote probes (TCP/IP, SMB, HTTP, NTP, SNMP, etc), it was possible to gather one or more fingerprints from the remote system. While the highest-confidence result was reported in plugin 11936, "OS Identification", the complete set of fingerprints detected are reported here.

Solution:-

Limit the exposure of system information by configuring firewalls, disabling unnecessary services, and restricting access to sensitive network protocols. Network administrators should also implement proper security controls such as intrusion detection/prevention systems (IDS/IPS) and keep operating systems and services updated with the latest security patches.

BusinessImpact:-

If attackers are able to identify the operating system running on a system, they may use this information to search for known vulnerabilities related to that OS. This can increase the risk of targeted attacks, exploitation, or unauthorized access to the system. Proper security configurations help reduce information disclosure and strengthen overall system security.

6.Vulnerability Name: - 22964-Service Detection

Severity:- INFO

Plugin:- 22964

Port:- 80

Description:-

Nessus was able to identify the remote service by its banner or by looking at the error message it sends when it receives an HTTP request.

Solution:-

Configure the server to disable or limit service banner information that reveals details about the running service. This can be done by modifying server configuration settings to hide version details and error messages. Additionally, ensure that the server and its services are regularly updated and patched to maintain security.

BusinessImpact:

If service details are exposed through banners or error messages, attackers may use this information to identify the technology or software running on the server. This can help them search for known vulnerabilities and plan targeted attacks, which may increase the risk of system compromise.

7.Vulnerability Name: - 10287-Traceroute Information

Severity:- INFO

Plugin:- 10287

Port:- 0

Description

Makes a traceroute to the remote host.

Solution:-

Configure network devices such as firewalls and routers to restrict or filter ICMP and traceroute requests from unauthorized sources. Limiting the response to such network probing techniques helps reduce the amount of network path information exposed to external users.

BusinessImpact:-

Traceroute information can reveal details about the network path and infrastructure between the source and the target system. Attackers may use this information to understand the network structure and identify potential targets or weak points, which could increase the risk of network-based attacks.

8.Vulnerability Name: - 10386-Web Server No 404 Error Code Check

Severity:- INFO

Plugin:- 10386

Port:- 80

Description:-

The remote web server is configured such that it does not return '404 Not Found' error codes when a nonexistent file is requested, perhaps returning instead a site map, search page or authenticationpage.

Nessus has enabled some counter measures for this. However, they might be insufficient. If a great number of security holes are produced for this port, they might not all be accurate.

Solution:-

Configure the web server to return the correct HTTP status codes, especially the 404 Not Found error when a non-existent page or file is requested. Proper server configuration helps ensure accurate responses and improves the effectiveness of vulnerability scanning tools.

BusinessImpact:-

If the web server does not return proper 404 error codes, security scanners may produce inaccurate or false vulnerability results. This can make it difficult for security teams to correctly identify real issues, potentially delaying proper risk assessment and security management.

Stage-3

Report:-

Title: - Implementing Zero Trust Network Access (ZTNA) with Zscaler: Securing User-to-Application Connectivity

Zero Trust Network Access (ZTNA)

Zero Trust Network Access (ZTNA) is a modern security model that allows users to securely access an organization's internal applications and data based on strict access control

policies. Unlike traditional Virtual Private Networks (VPNs), which allow users to access the entire network once connected, ZTNA follows the principle **“never trust, always verify.”**

In this approach, users are granted access only to the specific applications they require and only after their identity and device security have been verified.

ZTNA plays an important role in modern cybersecurity architecture. It removes the old concept of a trusted internal network and an untrusted external network, often called the **“castle-and-moat model.”** Instead, ZTNA hides the internal network from the internet and creates secure one-to-one encrypted connections between users and applications. This greatly reduces the chances of cyberattacks and prevents attackers from moving across the network.

The ZTNA Access Cycle

The ZTNA access cycle is a continuous process that verifies users and securely connects them to applications while enforcing security policies for every request.

1. User Request

A user from any location such as the office, home, or a public network attempt to access a specific internal application like a student portal, finance system, or internal server.

2. Identity and Device Verification

Before allowing access, the ZTNA system verifies the user’s identity using authentication methods such as multi-factor authentication (MFA). It also checks the security condition of the user’s device, ensuring that antivirus software, firewall protection, and other security controls are active.

3. Policy Enforcement

The ZTNA system checks the organization’s access policies to determine whether the user, device, and request context meet the requirements to access the application.

4. Establish Secure Connection

If the policy requirements are satisfied, the ZTNA solution creates an encrypted connection called a micro-tunnel between the user’s device and the requested application. The user gains access only to that specific application and not to the entire network.

5. Continuous Monitoring

The system continuously monitors the connection to detect unusual behaviour or security risks. If any suspicious activity is detected, the system can terminate the connection immediately.

6. Logging and Analytics

All access attempts, whether successful or unsuccessful, are recorded and analysed. These logs help in security monitoring, auditing, and improving access policies.

Zscaler (Zscaler Internet Access / Zscaler Private Access)

Zscaler is a cloud-based security platform that delivers security services directly from the cloud instead of relying on traditional on-premises hardware such as VPN devices.

For implementing ZTNA, Zscaler provides a service called **Zscaler Private Access (ZPA)**. ZPA enables secure and seamless access to private applications based on user identity without connecting users directly to the internal network.

Zscaler collects information from multiple sources, including:

1. Identity Providers (IdP)

Identity providers such as Azure Active Directory, Okta, or Ping Identity verify the user's identity during authentication.

2. Device Posture Partners

Security tools like CrowdStrike, Tanium, or Microsoft Defender check the health and security status of the user's device.

3. Business Applications

These include internal applications such as web portals, file servers, or legacy systems hosted in data centres or cloud platforms like AWS and Azure.

4. Zscaler App Connectors

App Connectors are lightweight software components installed in the organization's data centre or cloud environment. They establish secure outbound connections to the Zscaler cloud and make internal applications invisible to the internet.

Zscaler enforces security policies based on several principles:

Identity-Based Access

Access policies are defined based on user identity rather than IP addresses.

Application Segmentation

Users can only access specific applications instead of the entire network.

Least Privilege Access

Users are provided with the minimum level of access necessary to perform their tasks.

The Zscaler ZTNA Cycle

The Zscaler ZTNA cycle represents the real-world implementation of ZTNA using the Zscaler cloud platform.

1. User Login and Authentication

When a user tries to access an internal application, they are redirected to the organization's identity provider for authentication and multi-factor verification.

2. Device Posture Check

Zscaler integrates with endpoint security tools to verify that the device meets the organization's security requirements.

3. Policy Evaluation

The Zscaler cloud checks whether the user, device, location, and time meet the defined access policies.

4. Establishing the Micro-Tunnel

Once access is approved, Zscaler creates an encrypted micro-tunnel between the user's device and the application through the App Connector.

5. Traffic Inspection (Optional)

Zscaler may inspect network traffic for malware detection or data loss prevention if required.

6. Logging and Analytics

All user activities are logged in the Zscaler cloud platform for reporting, threat analysis, and compliance purposes.

MISP (Malware Information Sharing Platform) Integration

MISP is a threat intelligence platform that can enhance the security capabilities of Zscaler.

1. Threat Intelligence Feed

MISP collects and shares information about malicious IP addresses, domains, and URLs known as **Indicators of Compromise (IOCs)**.

2. Proactive Blocking

If a user attempts to access a malicious website listed in the MISP threat feed, Zscaler Internet Access can block the request before any harm occurs.

3. Enhanced Visibility

Security teams can use MISP together with Zscaler logs to analyse suspicious behaviour and identify possible threats.

Your College Network Information

The college network infrastructure consists of several components that support academic and administrative activities.

These include:

- Core routers and switches
- Distribution routers and switches
- Wireless access points
- Firewalls and Intrusion Detection Systems (IDS)
- Internal application servers hosting student information systems, research databases, and administrative portals
- A legacy VPN system used for remote access by faculty and staff

However, the current VPN model has several limitations.

Over-Permissioned Access

When users connect through VPN, they may gain access to the entire network, increasing the risk of internal attacks if their device is compromised.

Limited Access Control for External Users

External collaborators may require access to specific research applications without full network access.

Poor User Experience

Legacy VPN systems can be slow and require routing all internet traffic through the university network.

These limitations highlight the need for adopting Zero Trust Network Access.

Deploying ZTNA Using Zscaler in a College Network

To implement ZTNA using Zscaler, the organization must focus on securing application access rather than the entire network.

1. Application Discovery

All internal applications such as ERP systems, library portals, and research servers must be identified and documented.

2. User Segmentation

Users should be grouped based on roles such as students, faculty members, administrative staff, IT personnel, and external collaborators.

3. Deploying Zscaler App Connectors

App Connector virtual machines are installed in the data centre to securely connect internal applications to the Zscaler cloud.

4. Identity Provider Integration

Zscaler is integrated with an existing identity management system such as Microsoft Azure AD to manage authentication and access policies.

5. Access Policy Configuration

Security policies are defined to determine which users can access which applications and under what conditions.

6. Deploying Zscaler Client Connector

Users install the Zscaler Client Connector on their devices, allowing secure access to private applications through the Zscaler cloud.

Threat Intelligence

In the environment of ZTNA and Zscaler, **threat intelligence** helps security teams understand who is attempting to access resources, what they are trying to access, and from which location. It provides deeper insight into potential security risks instead of relying only on basic identity verification.

Threat intelligence gathers information from several sources to identify suspicious behaviour and possible cyber threats.

Sources of Threat Intelligence

1. Zscaler Cloud Logs

Zscaler generates internal logs that record detailed information about user activity. These logs show which users are accessing specific applications, the location from which the access is made, and the type of device being used.

2. External Threat Intelligence Feeds (MISP)

External platforms such as MISP provide updated information about cyber threats. These feeds include details about malicious domains, harmful IP addresses, phishing campaigns, and command-and-control servers used by attackers.

3. Identity Provider Logs

Logs from identity providers can reveal suspicious login behaviour. For example, if a user account logs in from two distant locations within a very short time, it may indicate that the account has been compromised.

Uses of Threat Intelligence

1. Adaptive Access Control

If threat intelligence indicates that a login attempt originates from a high-risk location or device, Zscaler can apply stricter authentication methods or completely deny access.

2. Identifying Compromised Accounts

Security analysts can analyse Zscaler logs to detect unusual user activity, such as accessing applications that the user has never used before. This may indicate stolen credentials.

3. Blocking Malicious Communications

Threat intelligence feeds help Zscaler Internet Access block communication with known malicious websites, even if the request originates from a user inside the organization.

Incident Response (Adapted for ZTNA)

Incident response in a Zero Trust environment is faster and more efficient because access is tightly controlled and monitored. When a security issue is detected, the system allows quick identification and isolation of the threat.

1. Detection

The Security Operations Center (SOC) identifies suspicious activity within the network. For instance, a faculty account may attempt to download an unusually large amount of data from a research server during unusual hours. Zscaler logs provide complete information about the user, device, application accessed, and data transfer details.

2. Containment

Once suspicious activity is detected, the SOC team can immediately isolate the affected user or device through the Zscaler dashboard. Since ZTNA provides application-level access instead of network-level access, blocking the user instantly stops access to all applications and prevents the attacker from moving within the network.

3. Eradication

The compromised device is removed from the network and examined by the IT support team. The device undergoes malware scanning and cleaning to eliminate any malicious software.

4. Recovery

After the device is secured and the user’s password has been reset, access can be restored by re-enabling the Zscaler policy for that user.

5. Post-Incident Review

After resolving the incident, the security team reviews the logs to determine how the compromise occurred, such as through phishing or malware infection. Based on the findings, security policies may be updated to prevent similar incidents in the future.

Benefits of this Incident Response Approach

1. Fast Containment

Security administrators can take immediate action through the cloud management portal, reducing response time significantly.

2. Prevention of Lateral Movement

Attackers cannot move across the internal network because access is restricted to individual applications.

3. Detailed Investigation

Zscaler maintains comprehensive logs of all activities, allowing security teams to perform accurate forensic analysis.

4. Regulatory Compliance

The system ensures that access to sensitive information is strictly monitored and controlled, helping organizations meet data protection and compliance requirements.

Understanding the Tool (Zscaler ZPA)

Zscaler Private Access (ZPA) is a cloud-based service designed to implement Zero Trust Network Access principles. It ensures that users are connected directly to applications rather than gaining access to the entire network.

1. Application-Focused Security

ZPA focuses on protecting applications rather than network segments. Access policies are created based on specific applications rather than IP addresses or network ranges.

2. Hidden Network Infrastructure

ZPA uses App Connectors within the organization’s data centre to create outbound connections to the Zscaler cloud. Since no inbound connections are required, internal servers remain invisible to external attackers.

3. Dark Cloud Concept

From an external perspective, the organization’s applications cannot be discovered through scanning or probing. Only authenticated and authorized users can see and access these applications.

4. Improved User Experience

Users do not need to manually connect to complex VPN clients. The Zscaler Client Connector operates in the background and automatically provides secure access to applications. Some web applications can even be accessed directly through a browser without installing any software.

5. High Scalability

As a cloud-based solution, ZPA can easily support thousands of users simultaneously without requiring hardware upgrades, unlike traditional VPN systems.

Conclusion

This project explored the application of modern cybersecurity techniques to protect web applications and organizational networks.

The **first stage** focused on **web application testing**, which plays a critical role in identifying vulnerabilities and ensuring that web-based systems remain secure and reliable. Understanding the testing process helped highlight common weaknesses that attackers may exploit.

The **second stage** examined **Nessus vulnerability reports**, which provide detailed information about security weaknesses in systems and networks. Analysing these reports helps organizations identify potential threats and implement effective mitigation strategies.

The **third stage** introduced **Zero Trust Network Access (ZTNA)** using **Zscaler Private Access (ZPA)**. This approach represents a major shift from traditional VPN-based access to a more secure, identity-driven model where users are connected directly to applications rather than the entire network.

By hiding internal applications from the internet and enforcing strict access policies, ZTNA significantly reduces security risks such as unauthorized access and lateral movement within the network.

Overall, the project provided valuable insights into modern cybersecurity practices and demonstrated how organizations can strengthen their security posture using advanced security frameworks and tools.

Future Scope

Future of Web Application Testing

As web applications become more advanced and widely used, ensuring their security and reliability will become increasingly important. Organizations depend heavily on web applications for daily operations, making security testing essential.

Some key trends shaping the future of web application testing include:

Increased Emphasis on Security

With the rising number of cyberattacks targeting web platforms, organizations will place greater importance on identifying and fixing vulnerabilities through regular testing.

Adoption of Artificial Intelligence and Machine Learning

AI and ML technologies will be widely used to automate testing tasks such as generating test cases, detecting vulnerabilities, and identifying unusual patterns in application behaviour.

Growth of Cloud-Based Testing

Many organizations are migrating applications to cloud platforms. Cloud-based testing environments offer advantages such as scalability, flexibility, and cost efficiency.

Greater Focus on User Experience Testing

Ensuring that web applications are user-friendly and efficient will become a key part of the testing process.

Future Scope of the Testing Process

The software testing process will continue to evolve as software systems grow more complex.

Continuous Testing

Continuous testing integrates testing activities throughout the Software Development Life Cycle (SDLC), allowing developers to detect and fix issues earlier in the development process.

AI and ML Integration

Artificial intelligence will increasingly assist in detecting hidden patterns, automating test execution, and improving the accuracy of testing results.

Cloud-Based Testing Platforms

Cloud testing solutions will enable organizations to perform large-scale testing without investing heavily in physical infrastructure.

Increased Automation

Automated testing tools will allow organizations to run tests more efficiently and reduce the need for manual testing.

Future Scope of SOC / SIEM

The future of Zero Trust architecture and security monitoring systems will be shaped by emerging technologies and evolving cyber threats.

1. AI-Driven Zero Trust

Artificial intelligence will enable dynamic security policies that continuously learn user behaviour and automatically detect suspicious activity.

2. Integration with IoT and Edge Computing

Zero Trust security models will extend to Internet of Things devices and edge computing systems to secure machine-to-machine communication.

3. Identity-Focused Security

Identity verification methods will become more advanced, incorporating techniques such as behavioural biometrics and decentralized identity frameworks.

4. Integration with SASE Architecture

Zero Trust will become a fundamental part of the Secure Access Service Edge architecture, combining multiple security services into a unified cloud platform.

5. Protection of Critical Infrastructure

Industries such as healthcare, manufacturing, and energy will increasingly adopt Zero Trust frameworks to secure critical operational technologies.

6. Automated Threat Detection and Response

Future security systems will integrate automation tools that can detect threats, isolate affected systems, and initiate response procedures automatically.

Topics Explored

- Data Collection and Aggregation
- Data Standardization and Normalization
- Data Enrichment and Analysis
- Information Sharing and Distribution
- Integration with Security Tools
- Customization and Configuration
- Automation and Security Orchestration
- Community Collaboration
- Incident Response Support
- Threat Intelligence Feeds
- Security Operations Center (SOC) and SIEM

Tools Explored

- Nmap
- Metasploit
- Nessus
- Kali Linux
- OWASP ZAP
- Burp Suite

THE END